# Web and Mobile Computing 3

**Node.js, TypeScript and Express**

HTL Leonding

2026-09-27

# Table of contents

Introduction 3

**1 Node.js 4**
1.1 Tools 5
1.2 Visual Studio Code 6
1.3 Introduction to Node.js 7
1.4 Introduction to `npm` 10

**2 TypeScript and Express 14**
2.1 Introduction to TypeScript 15
2.2 Arrays and Functions 18
2.3 Asynchronous Programming and Files 23
2.4 HTTP, REST and Express 27
2.5 Objects, Types and Classes 32

# Introduction

This book collects the theory chapters of **Web and Mobile Computing 3** (WMC3) at HTL Leonding, school year 2026/27. It grows over the year, one part per unit, and each part presents the chapters of its unit in the order they are taught.

## Tools you need

- Visual Studio Code with the REST Client extension
- Node.js (LTS version) and npm
- Git and a GitHub account

# Part 1

# Node.js

# 1.1 Tools

Stuck or have a question?

Ask the AI tutor for this unit. It knows the chapters and exercises of this unit. You can ask in German or English.

▶ Ask the WMC tutor on app.novedu.at

Scan the code, or type this address into your browser:

`https://app.novedu.at/304irtv7iq`

## 1.1.1 Git

We will use Git to track changes on your source code.

### 1.1.1.1 Commit messages

Use the following commit messages to make sure your assignment progress is transparent at any time:

- `task X (started): short description`
  - if you have begun the task, but it is not yet meaningfully complete (less than ~30% of the task is done)
- `task X (unstable): short description`
  - if you have mostly completed the task (at least ~80% of the task is done), but you have not reliably verified everything yet
  - also if you find a bug in terms of a task you previously assumed to be "stable"
- `task X (stable): short description`
  - if you have completed and verified the task

**Please note:**

- *X* is the task number you have been working on.
- You can fix the latest commit message with following command:

```
git commit --amend -m "your fixed message"
```

- In case you have already pushed the commit, you need to execute following command afterwards:

```
git push --force origin main
```

## 1.1.2 GitHub

We will use GitHub for hosting our remote repositories.

### 1.1.2.1 Connecting Git with GitHub

GitHub supports different modes of authentication:

- Username and password with two-factor authentication
- Personal access token
- SSH key

# 1.2 Visual Studio Code

Stuck or have a question?

Ask the AI tutor for this unit. It knows the chapters and exercises of this unit. You can ask in German or English.

▶ Ask the WMC tutor on app.novedu.at

Scan the code, or type this address into your browser:

`https://app.novedu.at/304irtv7iq`

Go to the `Extensions` icon in the vertical task bar on the left side and search for extensions on the market place, or simply click on the links below.

## 1.2.1 Recommended Plugins

### 1.2.1.1 TypeScript Development

- Visual Studio Intellicode
- Error Lens
- Better Comments
- Prettier

### 1.2.1.2 Network Development

- REST Client
- Docker

### 1.2.1.3 Git

- GitLens

### 1.2.1.4 Documentation

- Markdown PDF

### 1.2.1.5 Nice to have

- VS Code Icons
- Color Themes
- TODO Highlight

# 1.3 Introduction to Node.js

Stuck or have a question?

Ask the AI tutor for this unit. It knows the chapters and exercises of this unit. You can ask in German or English.

▶ Ask the WMC tutor on app.novedu.at

Scan the code, or type this address into your browser:

`https://app.novedu.at/304irtv7iq`

## 1.3.1 What is JavaScript?

- **dynamically typed** programming language
- first version in **1995** (Netscape, Sun Microsystems)
  - **dynamic web pages** (=> client-side development)
  - Meanwhile also used for **server-side development**
- supports different programming philosophies:
  - object-oriented programming
  - functional programming

## 1.3.2 What is a JavaScript engine?

- software component which executes JavaScript code
- of course each web browser needs one:
  - Spider-Monkey (Firefox)
  - V8 (Chrome, Edge)
  - JavaScriptCore (Safari)

## 1.3.3 What is Node.js?

- a JavaScript runtime
  - executes JavaScript **outside** of a web browser
- main components:
  - V8 JavaScript engine
  - API

**Please note:** Node.js is neither a programming language nor a framework!

## 1.3.4 Node.js Installation

On nodejs.org you find two versions:

- long-term support (**LTS**)
  - stable version for production applications
  - critical bugs will be fixed for 30 months
- current
  - version with latest features
- See https://github.com/nodejs/Release

**Please note:** To avoid unnecessary problems, we will stick to the LTS version!

### 1.3.5 Hello World

Create the file `hello-world.js` with the following content:

```
console.log("Hello World!");
```

Afterwards open the directory of `hello-world.js` in a terminal and type the following command:

```
$ node hello-world.js
```

As output you will get:

```
> Hello World!
```

**Please note:** You don't have to type the **$** and you won't get a **>** in the output. These prefixes should just help you distinguish between terminal inputs and outputs. To make it clear, you have to type `node hello-world.js` and you will get `Hello World!` as the output.

In the future we will shorten the description of such an example like this:

```
$ node hello-world.js
> Hello World!
```

### 1.3.6 Applications

There are different use cases for Node.js:

- console applications
  - e.g. Angular CLI
- web applications
  - e.g. Netflix
- desktop applications
  - with Electron framework (Node.js + Chromium)
  - e.g. Visual Studio Code

### 1.3.7 Features

- fast
  - written in C / C++
  - uses Chrome V8
- rich ecosystem
  - there exist lots of libraries (modules) you can use with Node.js
  - more of that later (see *npm*)
- full support of ES 2015 (ES 6)

### 1.3.8 What is ES?

- ECMAScript
  - basis of JavaScript
  - standardized by ECMA International
- ES is constantly adjusted
  - new language features added
  - yearly updates since 2015
- ES version features and browser compatibility
  - refer to http://kangax.github.io/compat-table/es2016plus/

In modern JavaScript projects, we usually use **ES modules** instead of CommonJS.

- CommonJS uses `require(...)`
- ES modules use `import ... from ...`

For this course, we use **ES modules** as the default approach. This keeps the syntax modern and consistent with current JavaScript usage.

# 1.4 Introduction to `npm`

Stuck or have a question?

Ask the AI tutor for this unit. It knows the chapters and exercises of this unit. You can ask in German or English.

▶ Ask the WMC tutor on app.novedu.at

Scan the code, or type this address into your browser:

`https://app.novedu.at/304irtv7iq`

## 1.4.1 What is `npm`?

- Formally called *Node Package Manager*
- Consists of three components:
  - website
    - description of available packages
  - registry
    - contains all packages
  - client app
    - Command Line Interface (CLI) for managing packages
    - automatically installed with `Node.js`

## 1.4.2 What do you do with `npm`?

- Manage your project
  - author, version number, license, …
- Manage your project dependencies
  - install / uninstall / update packages
- Publish your own project to the `npm` registry

## 1.4.3 Project dependencies

The following problems have to be solved in every project that depends on libraries:

- How do we define which dependencies the application needs?
- How do we know if dependencies should be updated?
- How do we reinstall dependencies on another machine?

`npm` solves those problems with the file `package.json`.

## 1.4.4 Create a `package.json`

- Open the project directory in your terminal.
- Run the following command:

```
$ npm init
```

- As a result, a file called `package.json` will be created:

```
{
  "name": "npm-demo",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "author": "John Snow",
  "license": "ISC"
}
```

### 1.4.5 Install the `chalk` package

- chalk is a package for formatting your console output.
- You can install it via `npm`:

  ```
  $ npm install chalk
  ```

  The exact version depends on the current npm registry. At the time this material was updated, the latest release is `6.0.0`, so the version shown below may differ slightly in your environment.

- You can check the installed `chalk` version:

  ```
  $ npm list
  > npm-demo@1.0.0 /home/mh/node/npm-demo
  >   chalk@6.0.0
  ```

- You will also find a new dependency in `package.json`:

  ```
  {
    "name": "npm-demo",
    "version": "1.0.0",
    "description": "",
    "main": "index.js",
    "type": "module",
    "scripts": {
      "test": "echo \"Error: no test specified\" && exit 1"
    },
    "author": "John Snow",
    "license": "ISC",
    "dependencies": {
      "chalk": "^6.0.0"
    }
  }
  ```

### 1.4.6 Semantic versioning

- Version number format: `MAJOR.MINOR.PATCH`
- `6.0.0` means
  - `MAJOR = 6`
  - `MINOR = 0`
  - `PATCH = 0`
- Increment:
  - `MAJOR` when you make incompatible API changes
  - `MINOR` when you add functionality in a backwards compatible way
  - `PATCH` when you make backwards compatible bug fixes
- More details: semver.org

### 1.4.7 Versions in `package.json`

- `6.0.0` means
  - install exactly version `6.0.0`

- `~6.0.0` means
  - install the most recent `6.0.x` version
- `^6.0.0` means
  - install the most recent `6.x.x` version

### 1.4.8 Install a specific `chalk` version and update

- You can uninstall `chalk` via npm:

```
$ npm uninstall chalk
```

- You can install a specific version (for example version `5.1.0`):

```
$ npm install chalk@5.1.0
```

- After that, you can update to a later version within the same major release:

```
$ npm update chalk
```

- This would update `chalk` to version `5.6.2` as of today
  - by changing the `node_modules` and `package-lock.json`
  - while respecting the restrictions in `package.json`

### 1.4.9 Import the installed `chalk` package

- Make sure the type is set to `module` in your `package.json` file.

```
"type": "module"
```

- Create a file `index.js` with the following content:

```
import chalk from "chalk";

console.log(chalk.red("Hello world"));
```

- Afterwards execute the program:

```
$ node index.js
```

### 1.4.10 Install and execute the `cowsay` package

- cowsay is a package for showing a configurable talking cow.
- `npm install cowsay`
  - installs the package to `node_modules/cowsay`
  - installs the executable to `node_modules/.bin`
- You can execute it with the following command:

```
$ ./node_modules/.bin/cowsay "Hello world!"
```

- An easier way is to use `npx`:

```
$ npx cowsay "Hello world!"
```

### 1.4.11 What is in `node_modules`?

- All packages are installed in a folder called `node_modules`
  - … and their dependencies
  - … and the dependencies of the dependencies …
- You can take a look at all installed packages:

```
$ npm list --all
```

- `node_modules` can contain a lot of files

  - don't copy it from machine to machine
  - never (!!!) add it to a Git repository

### 1.4.12 Git ignore `node_modules`

- Create a file called `.gitignore` in your Git repository.
  - the file must be named exactly like that
  - do not call it something like `.gitignore.txt`
- Add the following line to `.gitignore`:
  `node_modules/`

### 1.4.13 `package-lock.json`

- `npm install` also creates the file `package-lock.json`.
- The exact versions that were downloaded into `node_modules` are written to `package-lock.json`.
- Add `package-lock.json` to your Git repository.
- After that, your teammates can pull the repo and call `npm install`.
  - they will get the same package versions as you
  - even if a newer version is already available

### 1.4.14 Common problems

- `node` or `npm` is not recognized:
  - check whether Node.js is installed correctly
  - restart the terminal after installation
- `Cannot find module`:
  - check whether the package was installed with `npm install`
  - check whether the project uses the correct `package.json`
- `SyntaxError: Cannot use import statement outside a module`:
  - add `"type": "module"` to `package.json`

### 1.4.15 Check yourself

Train for Node.js and npm with the practice quiz on Node.js and npm. Each attempt asks 6 short questions, so you can take it as often as you like. The quiz is anonymous: it is for your own training, and your teacher cannot see who gave which answer.

Practice quiz on Node.js and npm

▶ Take the quiz on app.novedu.at

Scan the code, or type this address into your browser:

`https://app.novedu.at/xx62p6krvt`

# Part 2

# TypeScript and Express

# 2.1 Introduction to TypeScript

Stuck or have a question?

Ask the AI tutor for this unit. It knows the chapters and exercises of this unit. You can ask in German or English.

▶ Ask the WMC tutor on app.novedu.at

Scan the code, or type this address into your browser:

`https://app.novedu.at/uc8xynhvzp`

You have written JavaScript before, for plain web pages. This chapter is about what TypeScript adds on top of it, and about the handful of rules that decide whether the type system helps you or gets in your way.

## 2.1.1 Why TypeScript?

JavaScript is **dynamically typed**. A variable can hold anything, and a typo in a property name is only discovered when that line of code actually runs — possibly in production.

```
const user = { name: "Ada", age: 36 };
console.log(user.nmae);   // undefined - no error, no warning
```

TypeScript is a **superset** of JavaScript: every valid JavaScript program is a valid TypeScript program. It adds a type system that is checked **before** the program runs.

```
const user: { name: string; age: number } = { name: "Ada", age: 36 };
console.log(user.nmae);   // compile error: Property 'nmae' does not exist
```

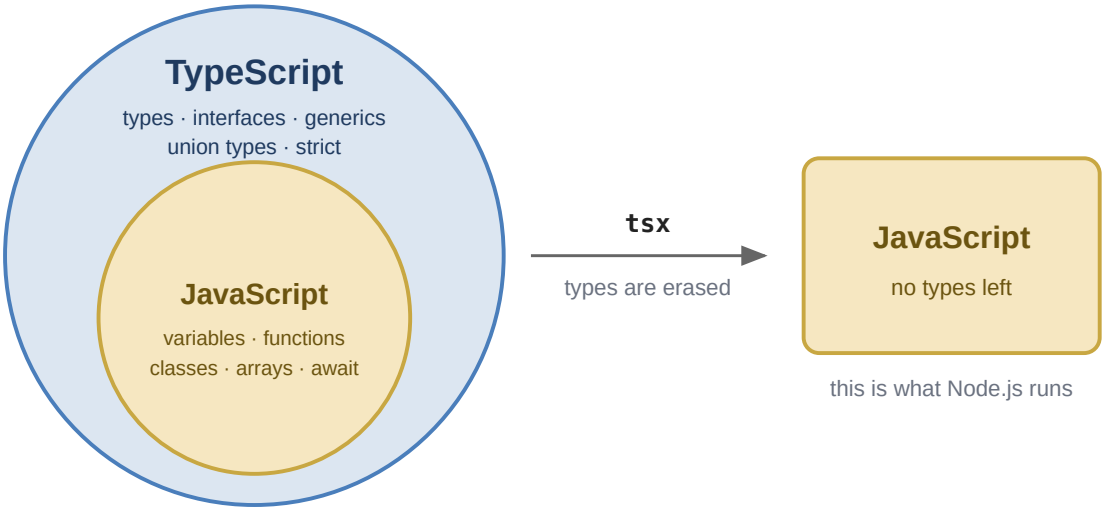



Figure 2.1.1: JavaScript sits inside TypeScript: TypeScript adds types, interfaces, generics, union types and strict mode, and after tsx has run only plain JavaScript is left

Everything you already know about JavaScript still applies. What you add is the description of what the values are — and that description is gone again by the time the program runs.

### 2.1.2 The most important sentence of this unit

**Types exist at compile time only. At runtime there is nothing left but JavaScript.**

TypeScript is *transpiled* to JavaScript, and during that step all type information is erased. There is no `typeof` check against your interfaces, no reflection, no casting that verifies anything.

```
const data = JSON.parse(text) as Article;   // compiles fine
console.log(data.price.toFixed(2));         // crashes if the JSON had no price
```

This matters enormously as soon as data enters the program from outside — a file, an HTTP request, a database. **You have to validate it yourself.** We will do this in every exercise of this unit.

### 2.1.3 Ten rules of the type system

Keep this table close for the first few weeks. Most of the confusion in this unit comes from one of these ten lines.

| | TypeScript |
|---|---|
| Type compatibility | **structural** – the shape decides, not the name ("duck typing") |
| Types at runtime | **none** – they are erased during transpilation |
| Type inference | works almost everywhere; write types only where they help |
| "no value" | `null` **and** `undefined` – prefer `undefined` |
| Untyped escape hatch | `any` (avoid it) and `unknown` (the safe one) |
| One of several fixed values | a **union of literal types**: "a" \| "b" |
| Operator overloading | does not exist – write `compareTo`, `equals`, … |
| Constants in a class | only `static readonly` |
| Generics at runtime | erased, like all other types |
| What is checked, and when | the compiler checks; the running program is plain JavaScript |

The two lines to remember: **structural typing** and **no types at runtime**.

### 2.1.4 Basic types

```
let done: boolean = false;
let count: number = 42;          // one numeric type only - no int/long/double
let name: string = "Ada";        // ' and " are equivalent, ` allows interpolation
let nothing: undefined = undefined;
let empty: null = null;
```

Type **inference** means the annotation is usually unnecessary:

```
let count = 42;              // inferred as number
count = "seven";             // compile error
```

Annotate **function parameters and return types**, and leave local variables to inference.

```
function toCelsius(fahrenheit: number): number {
  return (fahrenheit - 32) / 1.8;
}
```

#### 2.1.4.1 Template strings

```
const city = "Linz";
console.log(`It is ${toCelsius(72).toFixed(1)} °C in ${city}`);
```

### 2.1.5 `any` and `unknown`

`any` switches the type checker off for that value. It is contagious and almost always the wrong answer:

```
const data: any = JSON.parse(text);
data.whatever.at.all;     // compiles - crashes at runtime
```

`unknown` is the safe variant: you may hold the value, but you must narrow it before use.

```ts
const data: unknown = JSON.parse(text);
if (typeof data === "object" && data !== null && "name" in data) {
  // only here TypeScript lets you access data.name
}
```

### 2.1.6 Union types and literal types

When a value may only be one of a few fixed alternatives, TypeScript usually uses a union of **string literal types** rather than an `enum`:

```ts
type Stage = "main" | "tent" | "club";

let stage: Stage = "main";
stage = "garden";            // compile error
```

Advantages over `enum`: the value in the JSON file *is* the value in the code, no mapping needed, and it disappears completely at compile time.

Narrowing a union is called a **type guard**:

```ts
const STAGES = ["main", "tent", "club"] as const;

function isStage(value: string): value is Stage {
  return (STAGES as readonly string[]).includes(value);
}
```

That function is exactly the validation the compiler cannot do for you. You will write one in every exercise.

### 2.1.7 `strict` mode

The `@tsconfig/next` base configuration enables `strict`. The two rules that will bite you:

- **`strictNullChecks`** – a `string` can never be `null` or `undefined`. If a value may be absent, say so: `string | undefined`.
- **`noImplicitAny`** – a parameter without a type annotation is an error.

Do not turn these off. They are the reason for using TypeScript in the first place.

### 2.1.8 Toolchain

| Tool | Purpose |
|---|---|
| `tsc` | the TypeScript compiler: `.ts` → `.js` |
| `tsx` | runs a `.ts` file directly, without a visible compile step |
| `tsconfig.json` | compiler settings (target, module system, strictness, paths) |
| `vitest` | unit tests written in TypeScript |

In this unit we run everything with **`tsx`** and never call `tsc` by hand. We set a project up together in class — the steps are in the first exercise.

> ⚠ **`tsx` only transpiles**, it does not type-check. Your editor and `npx tsc --noEmit` do that. A program that runs is not necessarily a program that compiles.

# 2.2 Arrays and Functions

Stuck or have a question?

Ask the AI tutor for this unit. It knows the chapters and exercises of this unit. You can ask in German or English.

▶ Ask the WMC tutor on app.novedu.at

Scan the code, or type this address into your browser:

`https://app.novedu.at/uc8xynhvzp`

Arrays and functions are the two things you will touch in every program from here on. The concepts will look familiar; the notation will not. And there is one habit to unlearn: in TypeScript you work on arrays with **methods**, not with loops. Anything you would write as a loop with an `if` inside has a shorter, clearer form here.

## 2.2.1 Arrays

```
const names: string[] = ["Ada", "Grace", "Linus"];
const counts: Array<number> = [1, 2, 3];       // identical, rarely used
```

Arrays are **not** fixed size and not typed at runtime:

```
names.push("Alan");
console.log(names.length);   // 4
```

`const` protects the *binding*, not the contents. You cannot point the name at a different array, but you can still change the array it points at:

```
const list = [1, 2];
list.push(3);        // allowed
list = [4, 5];       // compile error
```

### 2.2.1.1 Useful basics

```
const first = names[0];
const last = names.at(-1);              // undefined if empty
const exists = names.includes("Ada");
const index = names.indexOf("Grace");   // -1 if not found
const part = names.slice(1, 3);         // copy, original untouched
```

⚠ `names[99]` is `undefined` at runtime, but TypeScript types it as `string`. This is a known hole in the type system. Be careful with indexed access.

### 2.2.1.2 Spread and destructuring

```
const more = [...names, "Barbara"];              // copy plus one element
const merged = [...names, ...counts.map(String)];

const [head, ...tail] = names;                   // head: string, tail: string[]
const { name, age } = user;                      // object destructuring
```

The spread operator is the normal way to copy an array or object instead of mutating it.

### 2.2.1.3 Tuples

A fixed-length array with a type per position, useful when a function has to return two values at once:

```ts
type Entry = [key: string, value: number];
const entry: Entry = ["temperature", 21.5];
const [key, value] = entry;
```

## 2.2.2 Functions

```ts
function add(a: number, b: number): number {
  return a + b;
}
```

Optional and default parameters:

```ts
function greet(name: string, greeting: string = "Hello", loud?: boolean): string {
  const text = `${greeting}, ${name}!`;
  return loud ? text.toUpperCase() : text;
}
```

> There is **no method overloading**: a function name exists exactly once, and a second declaration replaces the first. Use optional parameters, or a union type for the parameter, when you need to accept more than one shape of input.

### 2.2.2.1 Functions are values

A function is a value like any other: it can be stored in a variable, passed as an argument and returned from another function. The type of a function is written as its parameter list plus `=>` plus its return type.

```ts
type Formatter = (value: number) => string;

const asEuro: Formatter = (value) => `${value.toFixed(2)} €`;
```

### 2.2.2.2 Arrow functions

```ts
const double = (x: number): number => x * 2;       // expression body, implicit return
const log = (msg: string): void => {               // block body, explicit return
  console.log(msg);
};
```

Use arrow functions for everything you pass to an array method — that is what they are for.

## 2.2.3 The array methods

Three methods do most of the work. They differ in **how many elements come out**:

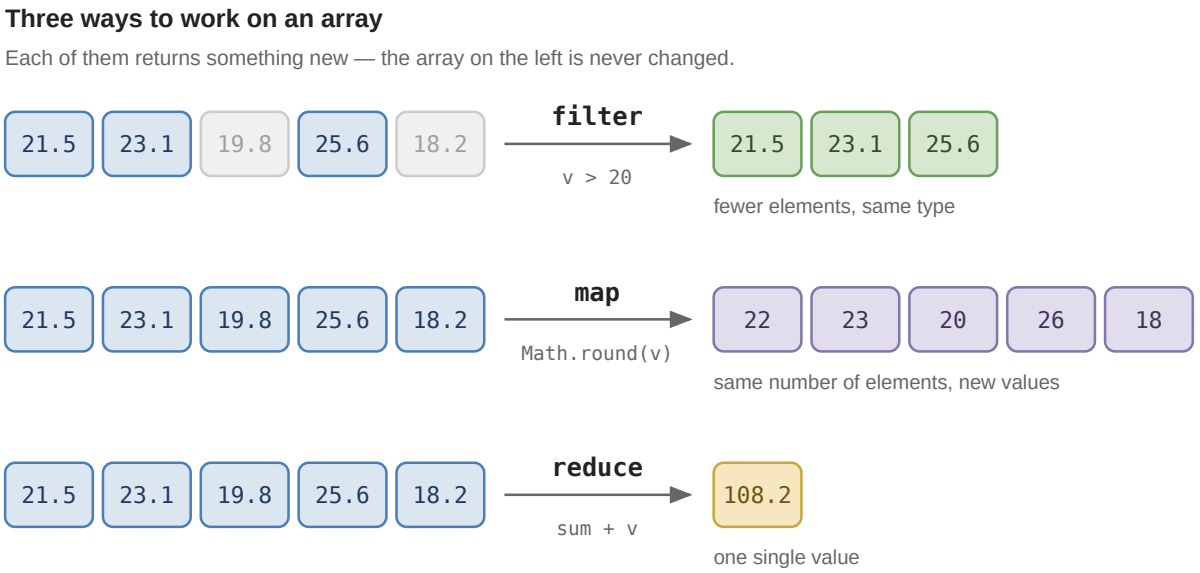



Figure 2.2.1: filter keeps some elements, map turns every element into a new one, reduce produces a single value

The rest of the table is variations on those three. All of these methods take a function as an argument (**higher-order functions**) and none of them modify the original array — they return a new one.

| You want | TypeScript | Result |
|---|---|---|
| keep only some elements | `filter(x => …)` | fewer elements, same type |
| turn every element into something else | `map(x => …)` | same count, different type |
| boil the array down to one value | `reduce(…)` | a single value |
| find out whether *any* element fits | `some(x => …)` | `boolean` |
| find out whether *all* elements fit | `every(x => …)` | `boolean` |
| get the first matching element | `find(x => …)` | element or `undefined` |
| count the matching elements | `filter(x => …).length` | `number` |
| sort | `toSorted((a, b) => …)` | a sorted copy |
| sum | `reduce((sum, x) => sum + x, 0)` | `number` |
| copy the array | `[...source]` | a new array |
| index the elements by a key | `new Map(source.map(x => [key, x]))` | `Map<K, V>` |
| group the elements by a key | `reduce` into a `Map` (see below) | `Map<K, V[]>` |

The three that matter most are **`filter`**, **`map`** and **`reduce`**. Almost everything else is a special case of one of them.

```
const readings = [
  { sensor: "outdoor", value: 21.5 },
  { sensor: "indoor", value: 23.1 },
  { sensor: "outdoor", value: 19.8 },
];

const outdoor = readings.filter((r) => r.sensor === "outdoor");
const values = readings.map((r) => r.value);
const total = values.reduce((sum, v) => sum + v, 0);
const average = total / values.length;
const warmest = readings.toSorted((a, b) => b.value - a.value)[0];
```

Note what `find` does when nothing matches: it returns `undefined` rather than throwing, and `strict` mode then forces you to handle that case before you can use the value.

#### 2.2.3.1 Sorting

`sort` compares **as strings** by default — `[10, 9, 1].sort()` gives `[1, 10, 9]`. Always pass a comparator:

```
numbers.toSorted((a, b) => a - b);                      // ascending numbers
people.toSorted((a, b) => a.name.localeCompare(b.name)); // by name
```

`toSorted` returns a **copy**; the older `sort` sorts in place. Prefer `toSorted`, and the same for `toReversed`.

#### 2.2.3.2 Grouping with `Map`

There is no ready-made method for grouping. The idiom is a `reduce` that builds up a `Map`, where each key holds the array of everything that belongs to it:

```
const bySensor = readings.reduce((map, r) => {
  const group = map.get(r.sensor) ?? [];
  group.push(r);
  return map.set(r.sensor, group);
}, new Map<string, typeof readings>());
```

A `Map` stores key/value pairs: `get`, `set`, `has`, `delete`, `size`, and it is iterable with `for (const [key, value] of map)`. Prefer it over a plain object whenever the keys are data rather than fixed property names.

#### 2.2.3.3 Chaining

```
const report = readings
  .filter((r) => r.value > 20)
  .map((r) => `${r.sensor}: ${r.value.toFixed(1)} °C`)
  .join(", ");
```

Each step produces a new array. That is fine for the data sizes in this unit and much easier to read than a loop.

### 2.2.4 Loops

Still available where they are genuinely clearer:

```
for (const name of names) { … }          // the VALUES - this is the one you want
for (const key in someObject) { … }      // the KEYS - rarely what you want
names.forEach((name, index) => { … });   // no break, no return
```

Use `for…of`. Use `for…in` almost never.

### 2.2.5 Two operators worth knowing

```
const port = config.port ?? 3000;        // null/undefined coalescing
const city = user?.address?.city;        // optional chaining -> undefined instead of a crash
```

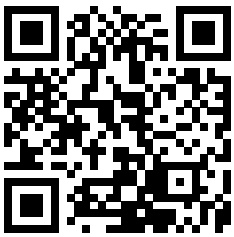
`??` reacts only to `null` and `undefined`, unlike `||` which also reacts to `0` and `""` — a classic source of bugs.

### 2.2.6 Check yourself

Train for this chapter and the previous one with the practice quiz on TypeScript basics, arrays and functions. Each attempt asks 6 short questions, so you can take it as often as you like. The quiz is anonymous: it is for your own training, and your teacher cannot see who gave which answer.

Practice quiz on TypeScript basics, arrays and functions

▶ Take the quiz on app.novedu.at



Scan the code, or type this address into your browser:

`https://app.novedu.at/mj3cyxyghi`

# 2.3 Asynchronous Programming and Files

Stuck or have a question?

Ask the AI tutor for this unit. It knows the chapters and exercises of this unit. You can ask in German or English.

▶ Ask the WMC tutor on app.novedu.at

Scan the code, or type this address into your browser:

`https://app.novedu.at/uc8xynhvzp`

Reading a file, calling a web service or querying a database takes time. This chapter is about what your program does in the meantime — and about the two keywords, `async` and `await`, that make waiting readable.

## 2.3.1 Node.js is single-threaded

**Node.js runs your JavaScript on a single thread.** There is no second thread picking up work in the background.

That sounds like a limitation, and for calculation-heavy work it is. But a web server does almost no calculation: it waits — for the disk, the database, another web service. While one request waits, the single thread is free to serve the next one. This is why Node.js is fast for I/O-bound servers and a bad choice for CPU-bound work.

The consequence: **never block the thread.** A blocking call does not slow down one request, it freezes the whole server for everyone.

```
import { readFileSync } from "fs";
import { readFile } from "fs/promises";

const bad = readFileSync("big.json", "utf-8");   // blocks the server. Never do this.
const good = await readFile("big.json", "utf-8"); // frees the thread while the disk works
```

We use `fs/promises` **exclusively** in this unit.

## 2.3.2 The `Promise`

A `Promise<T>` is a value that is not there yet. It is *pending*, then either *fulfilled* with a value or *rejected* with an error.

```
async function loadCount(): Promise<number> {
  const text = await readFile("data.json", "utf-8");
  return JSON.parse(text).length;      // a returned value is wrapped in a Promise
}

const count = await loadCount();
```

Two rules:

- A function that uses `await` must be declared `async`.
- An `async` function **always** returns a `Promise`, whatever you write in the return type.

### 2.3.2.1 Forgetting `await`

```
const count = loadCount();          // Promise<number>, not a number
console.log(count);                 // Promise { <pending> }
console.log(count + 1);             // compile error - and it would be nonsense anyway
```

Nothing warns you at runtime — the program happily prints a `Promise` and carries on. The type checker is your only protection here, which is another reason never to switch it off.

#### 2.3.2.2 Top-level `await`

Because we use ES modules, `await` works directly at the top level of a file. No `main()` wrapper is needed:

```
// src/app.ts
const data = await loadData();
console.log(data.length);
```

#### 2.3.2.3 Sequential vs. parallel

```
const a = await loadA();        // waits
const b = await loadB();        // ... and only then starts

const [a, b] = await Promise.all([loadA(), loadB()]);   // both at once
```

Use `Promise.all` whenever the calls do not depend on each other. Two requests that each take 200 ms then cost you 200 ms, not 400.

#### 2.3.2.4 Error handling

A rejected promise behaves like a thrown exception:

```
try {
  const text = await readFile(path, "utf-8");
} catch (error) {
  // error is of type `unknown` - you cannot just access error.message
  const message = error instanceof Error ? error.message : String(error);
  console.error(`Could not read ${path}: ${message}`);
}
```

⚠ In a `catch` block the error is `unknown`, not `Error`. Narrow it before use.

### 2.3.3 Reading files

```
import { readFile } from "fs/promises";

const text = await readFile("data/lineup.json", "utf-8");
```

**Always pass the encoding.** Without it you get a `Buffer` instead of a `string`.

#### 2.3.3.1 Paths

A relative path is resolved against the *working directory* — wherever `npm start` was called from, which is not necessarily where your source file lives. Build paths explicitly:

```
import { dirname, join } from "path";
import { fileURLToPath } from "url";

const dirName = dirname(fileURLToPath(import.meta.url));
const dataPath = join(dirName, "..", "data", "lineup.json");
```

💡 `__dirname` does not exist in ES modules. The three lines above are its replacement, and you will write them in every exercise from here on.

#### 2.3.3.2 JSON

```
const text = await readFile(dataPath, "utf-8");
const acts = JSON.parse(text) as Act[];      // a LIE the compiler believes
```

`JSON.parse` returns `any`. The `as Act[]` tells the compiler to stop asking questions — it verifies **nothing**. If the file is malformed, or a field is missing, the program crashes somewhere far away from the actual cause.

For file data we write ourselves, `as` is acceptable. For anything coming from a user or a network, validate it with a type guard, as in EX2.

#### 2.3.3.3 Writing files

```
import { writeFile } from "fs/promises";

const json = JSON.stringify(acts, null, 2);   // 2 = indentation, makes diffs readable
await writeFile(dataPath, json, "utf-8");
```

`writeFile` overwrites the whole file and creates it if it does not exist.

#### 2.3.3.4 CSV

There is no built-in CSV parser. Split the text yourself:

```
import { EOL } from "os";

const lines = text.split(EOL).filter((line) => line.trim().length > 0);
const rows = lines.slice(1);                  // skip the header
const tools = rows.map((line) => parseTool(line));
```

> ⚠ Use `os.EOL`, never a hard-coded `"\n"` — the files in this repository are edited on Windows and on Linux. If a file may come from either, split with `/\r?\n/` instead.

### 2.3.4 Lazy loading and caching

Reading a file on every request is wasteful. Read it once, on first use, and keep it:

```
let acts: Act[] | null = null;              // null = not loaded yet

export async function getActs(): Promise<Act[]> {
  if (acts === null) {
    acts = await loadActsFromFile();
  }
  return acts;
}
```

Note the type `Act[] | null` — `strict` mode forces you to make "not loaded yet" explicit, and to handle it. This pattern shows up in EX3 and EX4.

### 2.3.5 Timers

```
setTimeout(() => console.log("later"), 1000);        // once, after 1 s
const handle = setInterval(() => tick(), 1000);      // every second
clearInterval(handle);                               // stop it - do not forget
```

`setTimeout` does **not** pause the program. The following line runs immediately; the callback runs later, when the thread is free.

```
console.log("a");
setTimeout(() => console.log("b"), 0);
console.log("c");
// prints a, c, b
```

Who does what, and when:

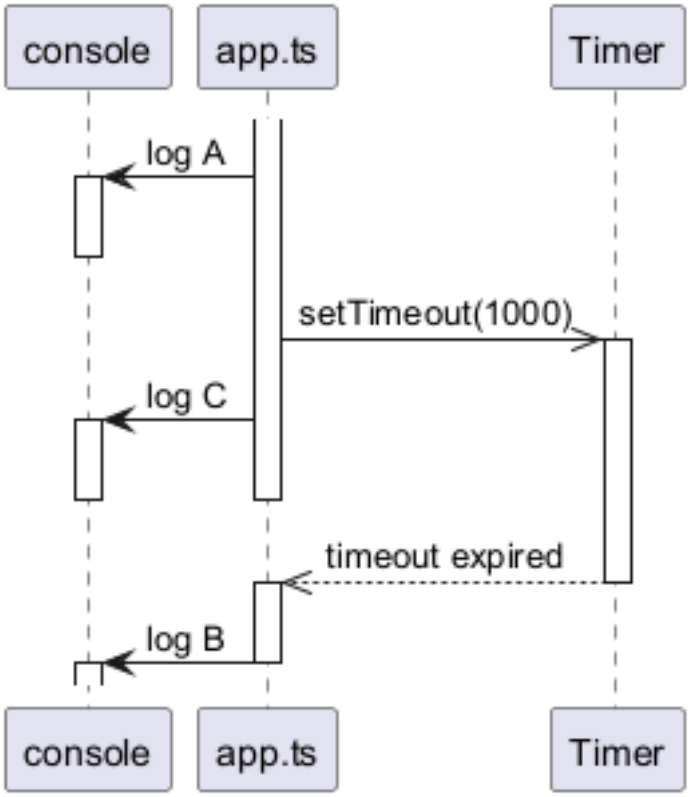



Figure 2.3.1: sequence diagram: app.ts logs A, hands the timer a callback, logs C, and only when the timer has expired does the callback log B

Your code hands the callback to the timer and **carries straight on**. The timer is not your thread, and the callback only gets its turn once your code has finished what it was doing.

# 2.4 HTTP, REST and Express

Stuck or have a question?

Ask the AI tutor for this unit. It knows the chapters and exercises of this unit. You can ask in German or English.

▶ Ask the WMC tutor on app.novedu.at

Scan the code, or type this address into your browser:

`https://app.novedu.at/uc8xynhvzp`

You have used HTTP from the browser side for years. This chapter is about the other end: what a request actually contains, and how to answer one with Express.

We build the server step by step, and test every step with the REST Client before adding the next one. One route that you have really called beats four routes that you have not.

## 2.4.1 HTTP in three minutes

A client sends a **request**, a server sends back a **response**. Both consist of a start line, headers and an optional body.

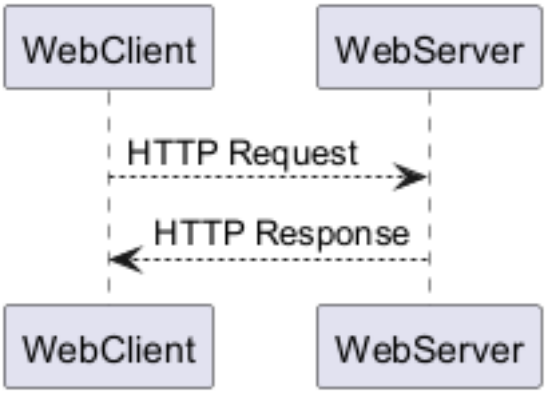



Figure 2.4.1: sequence diagram: a web client sends an HTTP request to a web server, which sends back an HTTP response

```
GET /api/acts?stage=main HTTP/1.1        request line: method, path, query, version
Host: localhost:3000                      headers
                                          (empty line)
                                          body - empty for GET

HTTP/1.1 200 OK                           status line
Content-Type: application/json            headers
                                          (empty line)
[{"id":"a1","name":"..."}]                body
```

### 2.4.1.1 Methods

| Method | Meaning | Body | Repeatable without side effects |
|---|---|---|---|
| `GET` | read | no | yes |
| `POST` | create | yes | **no** |

| Method | Meaning | Body | Repeatable without side effects |
|---|---|---|---|
| `PUT` | replace completely | yes | yes |
| `PATCH` | change partially | yes | yes |
| `DELETE` | delete | no | yes |

#### 2.4.1.2 Status codes

Using these correctly is a competence you are expected to have by the end of this unit.

| Code | Name | Use it when |
|---|---|---|
| `200` | OK | the request succeeded |
| `201` | Created | a `POST` created a resource — plus a `Location` header |
| `204` | No Content | success, but there is nothing to return (often `DELETE`) |
| `400` | Bad Request | the client sent nonsense (missing field, invalid value) |
| `404` | Not Found | the requested resource does not exist |
| `409` | Conflict | the request contradicts the current state (already lent out) |
| `500` | Internal Server Error | **your** code failed |

Rule of thumb: `4xx` is the client's fault, `5xx` is yours.

## 2.4.2 REST

A **resource** is a thing your API is about: an act, a tool, a loan. REST says: put the resource in the **path** (a noun, plural), and the verb in the **HTTP method**.

```
GET    /api/acts            all acts
GET    /api/acts/a1         one act
POST   /api/acts            create one
PUT    /api/acts/a1         replace one
DELETE /api/acts/a1         delete one

GET    /api/tools/t1/loans  the loans of one tool  (nested resource)
```

Not REST — the verb belongs in the method, not the path:

```
GET /api/getAllActs         no
GET /api/acts/delete/a1     no
POST /api/createAct         no
```

#### 2.4.2.1 Three ways to pass data

| | Looks like | Use for |
|---|---|---|
| **Route parameter** | `/api/acts/:id` | *which* resource — required, part of the identity |
| **Query parameter** | `/api/acts?stage=main&sort=time` | filtering, sorting, paging — optional |
| **Body** | JSON in the request body | the data of the resource, for `POST`/`PUT`/`PATCH` |

## 2.4.3 A first Express server

```
npm install express http-status-codes
npm install --save-dev @types/express nodemon
```

`src/app.ts`:

```
import express, { Request, Response } from "express";
import { StatusCodes } from "http-status-codes";

const app = express();

app.use(express.json());        // parses a JSON request body into req.body

app.get("/", (req: Request, res: Response) => {
  res.send("hello world");
});

app.listen(3000, () => console.log("Server listening on http://localhost:3000"));

npm run dev        # nodemon restarts on every save
```

`express.json()` is easy to forget. Without it `req.body` is `undefined` on every `POST`, and you will look for the bug in the wrong place.

## 2.4.4 Middleware

Express processes a request through a **chain** of functions. Each one may answer the request itself, or do a bit of work and hand it on to the next one with `next()`.

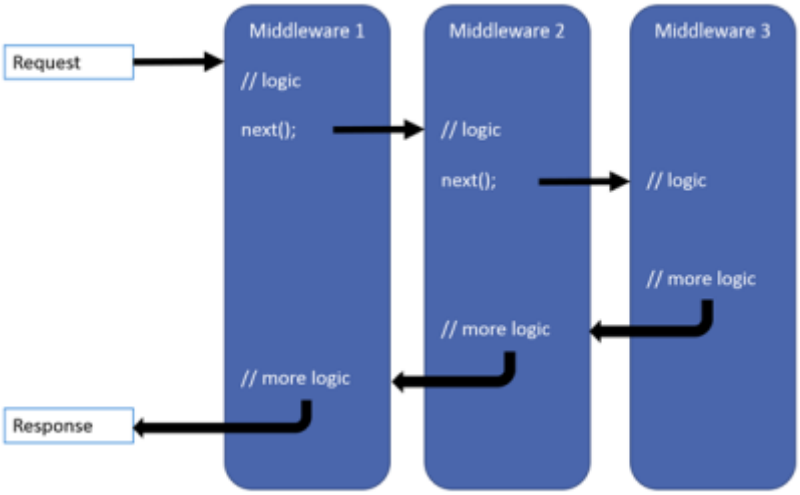



Figure 2.4.2: the request passes through middleware 1, 2 and 3, each running some logic and calling next(), and the response travels back out through the same chain in reverse

Note the way back: after the last one has run, the response travels out through the same chain in **reverse** order.

```
app.use((req, res, next) => {
  console.log(`${req.method} ${req.url}`);
  next();                       // forget this and the request hangs forever
});

app.use(express.static("public"));   // serve static files from ./public
app.use("/api/acts", actRouter);     // mount a router under a path prefix
```

**Order matters.** The first matching handler wins.

## 2.4.5 Routers

Do not put your endpoints in `app.ts`. One router per resource, in its own file:

```
// src/act/act-router.ts
import { Router, Request, Response } from "express";
import { StatusCodes } from "http-status-codes";
import { findAll, findById } from "./act-service";
```

```ts
export const actRouter = Router();

actRouter.get("/", async (req: Request, res: Response) => {
  const stage = req.query.stage;             // string | string[] | undefined !
  res.json(await findAll(typeof stage === "string" ? stage : undefined));
});

actRouter.get("/:id", async (req: Request, res: Response) => {
  const act = await findById(req.params.id);

  if (act === undefined) {
    res.status(StatusCodes.NOT_FOUND).json({ error: `No act with id ${req.params.id}` });
    return;                                  // do NOT forget the return
  }

  res.json(act);
});

// src/app.ts
app.use("/api/acts", actRouter);
```

Three things to note:

- `req.params.id` is **always** a `string`. Convert and validate it yourself.
- `req.query.stage` is `string | string[] | ParsedQs | undefined` — a client can send `?stage=a&stage=b`. Narrow it before use.
- After `res.status(...).json(...)` you must `return`. Express does not stop the handler for you, and sending two responses is a runtime error.

### 2.4.6 Architecture: router and service

`request → router → service → data (file, later: database)`

- The **router** speaks HTTP: read parameters, validate input, choose the status code. It contains no business logic.
- The **service** contains the business logic and loads/stores the data. It knows nothing about HTTP — no `Request`, no `Response`, no status codes.

That separation is what makes the database unit possible: there you replace the data access inside the service with SQLite, and the router does not change at all.

### 2.4.7 DTOs

The objects you send over the wire are **not** your domain objects. A `Tool` class has methods and maybe internal state; what goes into the JSON is a plain, flat object — a **Data Transfer Object**.

```ts
export type ToolDto = {
  id: string;
  name: string;
  category: Category;
  available: boolean;
};

export function toToolDto(tool: Tool): ToolDto {
  return { id: tool.id, name: tool.name, category: tool.category, available: tool.isAvailable };
}
```

Why bother: the DTO is your API's contract. It stays stable while the model behind it changes, and it lets you leave internal fields out.

### 2.4.8 Testing with the REST Client

Put a `requests.http` file next to your source and use the VS Code REST Client extension. `###` separates the requests, and each one gets a *Send Request* link.

```
@host = http://localhost:3000

### all acts
GET {{host}}/api/acts

### filtered
GET {{host}}/api/acts?stage=main&sort=time

### one act
GET {{host}}/api/acts/a1

### create a loan
POST {{host}}/api/loans
Content-Type: application/json

{
  "toolId": "t1",
  "borrower": "Ada Lovelace"
}
```

Keep this file up to date — in the exercises it is part of the hand-in.

### 2.4.9 Check yourself

Train for this chapter with the practice quiz on HTTP, REST and Express. Each attempt asks 6 short questions, so you can take it as often as you like. The quiz is anonymous: it is for your own training, and your teacher cannot see who gave which answer.

**Practice quiz on HTTP, REST and Express**

▶ Take the quiz on app.novedu.at

Scan the code, or type this address into your browser:

`https://app.novedu.at/cakqtl7rxi`

# 2.5 Objects, Types and Classes

Stuck or have a question?

Ask the AI tutor for this unit. It knows the chapters and exercises of this unit. You can ask in German or English.

▶ Ask the WMC tutor on app.novedu.at

Scan the code, or type this address into your browser:

https://app.novedu.at/uc8xynhvzp

You have met classes, inheritance and interfaces before. This chapter is about the parts TypeScript does its own way — and about when a class is the wrong tool and a plain type is the right one.

## 2.5.1 Object literals

In JavaScript an object is created without a class. It is a bag of key/value pairs:

```
const act = {
  id: "a01",
  name: "Kernel Panic",
  durationMinutes: 60,
};

console.log(act.name);
console.log(act["name"]); // the same thing
```

TypeScript infers the type of `act` from the literal. You can also name that type:

```
type Act = {
  id: string;
  name: string;
  durationMinutes: number;
  encore?: boolean; // optional -> boolean | undefined
  readonly createdAt: string; // may only be set once, when the object is created
};
```

This is the workhorse of the unit. Most “objects” in our APIs are plain data with a type — no class needed.

## 2.5.2 Structural typing (duck typing)

**The single most important rule of the type system.** TypeScript compares types by their *shape*, not by their *name*. Two types with the same members are the same type, even if they were declared separately and never mention each other.

```
type Point = { x: number; y: number };
type Coordinate = { x: number; y: number };

const p: Point = { x: 1, y: 2 };
const c: Coordinate = p; // fine. Same shape = same type.
```

Nothing has to `implement` anything. If it walks like a duck, it is a duck:

```
type Named = { name: string };

function greet(thing: Named) {
  console.log(`Hello ${thing.name}`);
}

const user = { name: "Ada", age: 36 };
```

```
greet(user); // fine - it HAS a name
greet(new Tool("t1", "Hammer")); // fine, if Tool has a `name` property
```

#### 2.5.2.1 Excess property check

There is one exception that confuses everyone once:

```
const named: Named = { name: "Ada", age: 36 }; // ERROR: 'age' does not exist in 'Named'

const thing = { name: "Ada", age: 36 };
const named2: Named = thing; // fine!
```

A **literal** assigned directly is checked strictly, because extra properties there are almost always a typo. A value that already has a type is only checked for compatibility.

### 2.5.3 `type` or `interface`?

Both describe a shape, and for objects they are nearly interchangeable.

| | `type` | `interface` |
|---|---|---|
| Objects | yes | yes |
| Unions (`"a" \| "b"`) | **yes** | no |
| Tuples, function types | yes | awkward |
| Can be re-opened / merged | no | yes |
| `extends` | via `&` | yes |

**Convention for this unit:** use `type` everywhere. Use `interface` only when you really want a contract that several classes implement. Do not prefix interface names with `I`; TypeScript does not do that.

### 2.5.4 Classes

The syntax will look familiar. Three details are worth memorising, because they differ from what you would expect.

```
export class Tool {
  // 1. no access modifier means PUBLIC - there is no "internal"
  readonly id: string;
  readonly name: string;
  private timesLent = 0; // type inferred from the initialiser

  static readonly MAX_LOAN_DAYS = 14; // 2. no `const` in a class - use `static readonly`

  constructor(id: string, name: string) {
    this.id = id;
    this.name = name;
  }

  // 3. `get` is a method with a keyword, and it is called WITHOUT parentheses
  get isPopular(): boolean {
    return this.timesLent > 10;
  }

  lend(): void {
    this.timesLent++;
  }
}

const tool = new Tool("t1", "Cordless drill");
console.log(tool.isPopular); // no ()
```

#### 2.5.4.1 Parameter properties

The constructor boilerplate above has a short form:

```
export class Tool {
  constructor(
    readonly id: string,
    readonly name: string,
    private timesLent = 0
  ) {}
}
```

The modifier in the parameter list declares the field **and** assigns it. There is no other short form for this, and you will see it in a lot of TypeScript code.

#### 2.5.4.2 `private` is not a security feature

```
class A {
  private secret = 42; // compile-time only - visible at runtime
  #reallySecret = 43; // truly private, enforced by the JavaScript runtime
}
```

`private` is erased with everything else during transpilation. `(a as any).secret` reads it without complaint. For our purposes `private` is fine — just do not mistake it for protection.

#### 2.5.4.3 Static factory methods

A constructor cannot fail gracefully, cannot be `async` and cannot have overloads. The usual answer is a **static factory**:

```
export class Tool {
  private constructor(readonly id: string, readonly name: string) {}

  /** Returns undefined instead of throwing when the line is malformed. */
  static fromCsvLine(line: string): Tool | undefined {
    const [id, name] = line.split(";");
    return id && name ? new Tool(id, name) : undefined;
  }
}
```

Note the `private constructor`: it forces everyone through the factory, so the validation cannot be skipped. You will use this pattern in EX4.

#### 2.5.4.4 Inheritance

```
class FullDayLoan extends Loan {
  constructor(id: string, toolId: string) {
    super(id, toolId); // must be called first
  }

  override describe(): string {
    // `override` is optional but recommended
    return `${super.describe()} (full day)`;
  }
}
```

`abstract class`, `abstract` members and `implements` all exist and behave as you expect. `instanceof` works at runtime, because classes — unlike interfaces and types — *do* survive transpilation.

We use inheritance sparingly in this unit. Prefer a union of types with a discriminating field:

```
type Loan =
  | { kind: "open"; toolId: string; since: string }
  | { kind: "returned"; toolId: string; since: string; until: string };

function describe(loan: Loan): string {
  // TypeScript narrows on `kind` - inside each branch the type is exact
  return loan.kind === "returned"
```

```
    ? `returned on ${loan.until}`
    : `open since ${loan.since}`;
}
```

This is a **discriminated union**, and it replaces a lot of small class hierarchies.

### 2.5.5 Classes vs. plain objects

| Use a **class** when | Use a **type** when |
|---|---|
| there is behaviour, not just data | it is pure data |
| there are invariants to protect | it comes from JSON or goes into JSON |
| you need `instanceof` | you need a DTO |

Remember from `03_async-and-files`:

```
const tools = JSON.parse(text) as Tool[]; // NOT Tool instances - no methods!
```

`JSON.parse` produces plain objects. `as Tool[]` only silences the compiler; calling a method on one of them crashes at runtime. If you want real instances, construct them:

```
const tools = (JSON.parse(text) as ToolDto[]).map((dto) => Tool.fromDto(dto));
```

This is the `toDto`/`fromDto` pattern, and it is the heart of EX4.

### 2.5.6 `Map` and `Set`

```
const byId = new Map<string, Tool>();
byId.set(tool.id, tool);
const found = byId.get("t1"); // Tool | undefined
byId.has("t1");
byId.delete("t1");
byId.size; // a property, not a method

for (const [id, tool] of byId) {
  // ...
}
const all = [...byId.values()];
```

A `Map` holds key/value pairs, a `Set` holds each value at most once. Build a `Map` from an array in one step:

```
const byId = new Map(tools.map((tool) => [tool.id, tool]));
```

> ⚠ A plain object (`{}`) also maps keys to values, but its keys are always strings, it inherits properties from `Object.prototype`, and it has no `size`. Use `Map` whenever the keys are **data**.

### 2.5.7 Equality

```
"1" == 1; // true  - == converts types. Never use it.
"1" === 1; // false - === compares type and value. Always use this.

({ a: 1 } === { a: 1 }); // false - reference comparison
```

There is no operator overloading, and no built-in convention for value equality. If you need it, write an `equals` method and call it explicitly.